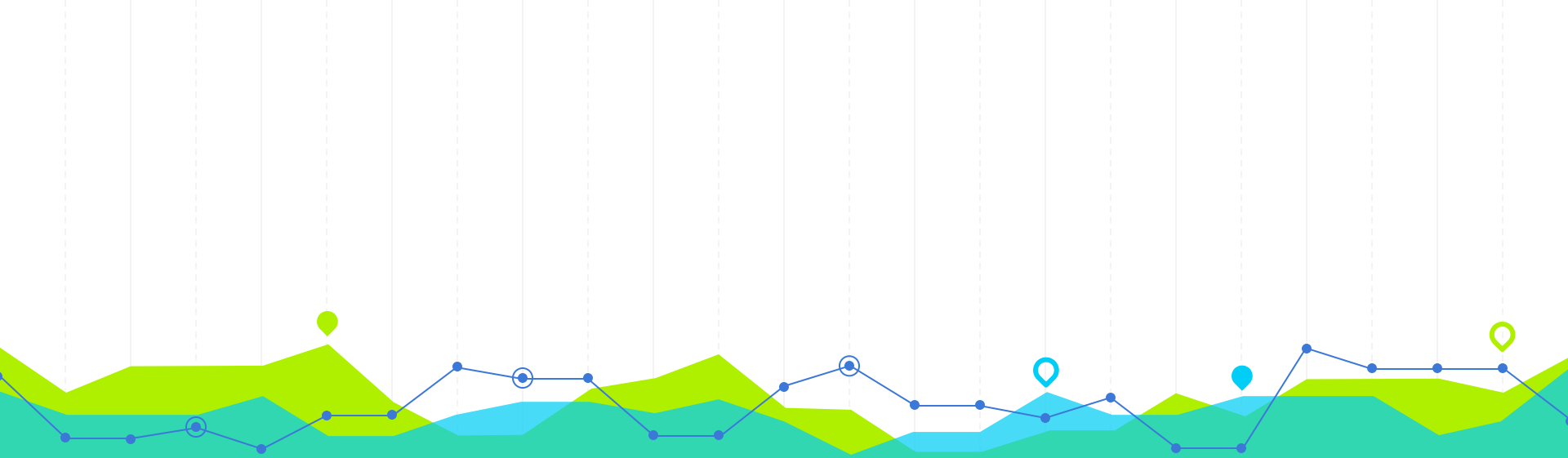




Coder en Python

pour la production

Le programme

Créer et coder un projet Python

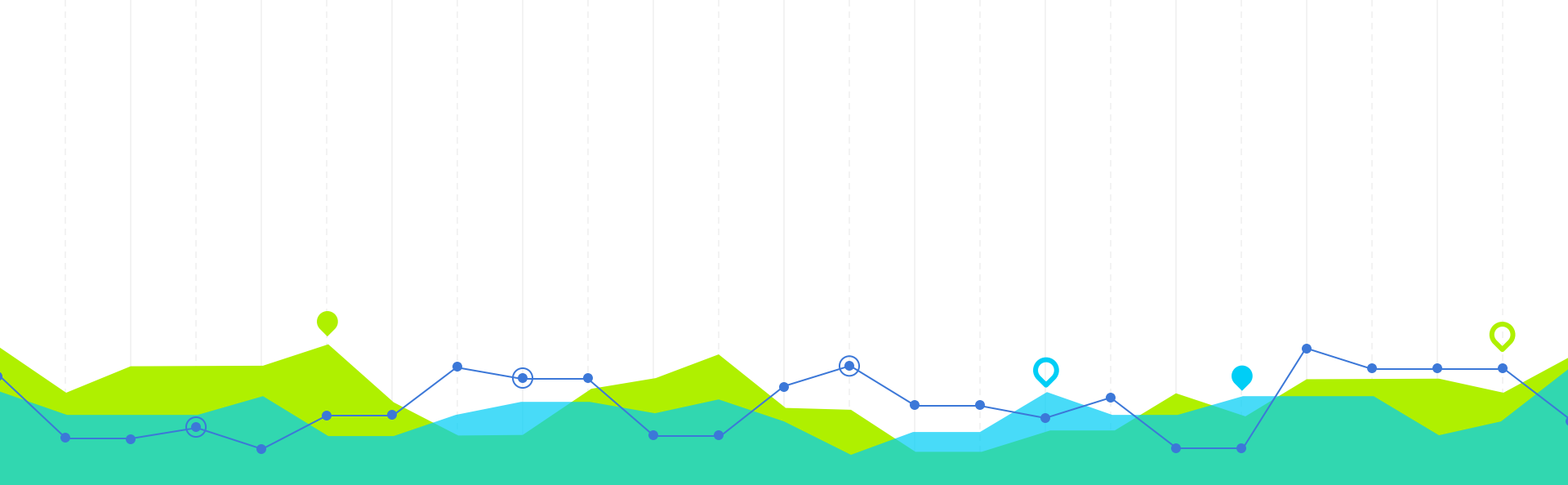
Objectifs :

- comprendre comment est structuré un projet python
- comprendre la Programmation Orientée Objet en python
- savoir utiliser Git en entreprise

Programme :

1. Structure d'un **projet Python**
2. POO en Python
3. Savoir utiliser **Git en CLI**
4. Savoir utiliser les **fonctionnalités** de GitHub / GitLab

	J2
13:30 - 15:00	Créer et coder dans un projet Python
15:00 - 15:15	
15:15 - 16:45	



**Créer et coder
dans un projet
Python**

Sommaire

1. Structure d'un **projet Python**
2. **POO** en Python
3. Savoir utiliser **Git en CLI**
4. Savoir utiliser les **fonctionnalités** de GitHub / GitLab



Objectif

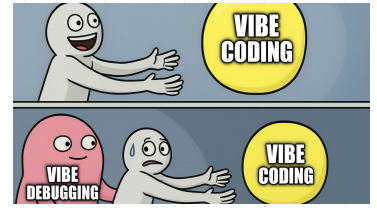
Créer et coder dans un projet Python

De bonnes habitudes pour coder en Python
= plus à même de travailler en équipe !



Pourquoi apprendre à coder en python ?

- Les assistants de code permettent de traduire vos demandes exprimées en langage naturel en code de grande qualité
- Capable de générer en quelques minutes un outil complet si les *prompts* sont bien formulés
- Important d'éviter l'effet boîte noire: vous êtes **garant du bon fonctionnement de votre application/projet**
- Plus vos **demandes seront précises**, plus les assistants fourniront des réponses adaptées
-> vous avez besoin de guider le LLM le plus possible pour éviter les boucles de rétroaction
- Eviter la dette technique qui pourrait être due à des mauvaises décisions des LLMs (technologies anciennes, duplication du code, secrets exposés dans le code en clair, etc.)



 Claude

Claude will return soon

Claude is currently experiencing a temporary service disruption.
We're working on it, please check back soon.

Try again

Coupure des LLMs principaux le
03/09/2026

Les assistants de code

- Les assistants de code permettent de **générer du code** via des appels à un LLM au sein d'un projet tout en ayant une compréhension globale des attentes du projet
- Ils peuvent **accéder au contexte de votre projet** (structure des modules, convention de code, nommage des variables) et venir modifier les parties d'intérêt selon un prompt
- Les assistants de code permettent d'**exposer certains outils utiles pour les projets**: systèmes multi-agents, tests, déploiements, review de code, documentation, résolution de bugs, etc.
- Très utiles également pour la compréhension globale d'un grand projet sur lequel vous souhaitez travailler (projet *opensource* ou nouveau projet dans une entreprise par exemple)
- Le **code source** de votre projet est **envoyé** à fournisseur de LLM, pensez donc toujours à vous assurer que le code que vous partagez n'est pas confidentiel !



Quelques exemple d'assistants de code



- **GitHub Copilot:** intégré par défaut dans VSCode (pas dans Codium), permet de faire de l'autocomplétion pendant l'édition, ~10\$/mois pour la version pro



- **Cursor:** issu d'un fork de VSCode, racheté par SpaceX en 2026 pour 60 milliards de \$, possibilité d'utiliser Claude, GPT, DeepSeek ou des modèles locaux



- **Claude Code:** utilisable en ligne de commande (CLI), probablement le plus utilisé aujourd'hui

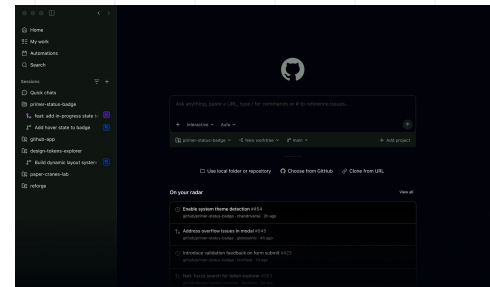


Vibe

- **Mistral vibe:** optimisé pour les modèles de mistral, proche de Claude Code



- **Codex:** développé par OpenAI en 2025, très similaire à Cursor, s'intègre bien avec les LLMs GPT



Screenshot de Copilot

Exemples d'utilisation pour de la production

Générer des tests	À partir d'une fonction pure ou d'un endpoint, demander des cas nominaux + bords + mocks	Couverture réelle, fixtures réalistes, pas de tests tautologiques
Faire un POC d'une API	Esquisse FastAPI + modèles Pydantic + routes CRUD	Auth, validation, codes HTTP, idempotence
CI / packaging	<code>pyproject.toml</code> , GitHub Actions, pre-commit (ruff, mypy)	Versions épinglées, caches, secrets CI
Expliquer un incident	Coller stack trace + extrait → hypothèses ordonnées	Reproduire localement avant de patcher
Revue assistée	"Quels risques dans ce diff ?" (race, N+1, fuites de connexions)	Ne remplace pas la review humaine
Refactoring ciblé	Extraire un repository, passer sync → async sur un client	Comportement inchangé + tests verts
Documentation	README d'un service, runbook, OpenAPI descriptions	Exactitude vs le code réel

AI IDE after I send "Still broken" for the 15th time



Outils externes intégrés aux assistants LLM

- Les assistants de code deviennent d'autant plus puissants que vous leur donnez accès à des **extensions**
- Accès aux fonctionnalités de votre terminal: lancer des commandes bash, pytest, ruff, uv, etc.
- **MCP**: pour brancher le LLM à des serveurs d'outils afin de faire des appels structurés (navigateur, base de données, GitHub, mails, Confluence, Notion, etc.)
- **Skills**: chargés dynamiquement quand le LLM en a besoin, fichiers qui permettent d'orienter le comportement de l'agent avec des prompts décrivant les spécificités attendues

```
---
name: my-skill-name
description: A clear description of what this skill does and when to use it
---

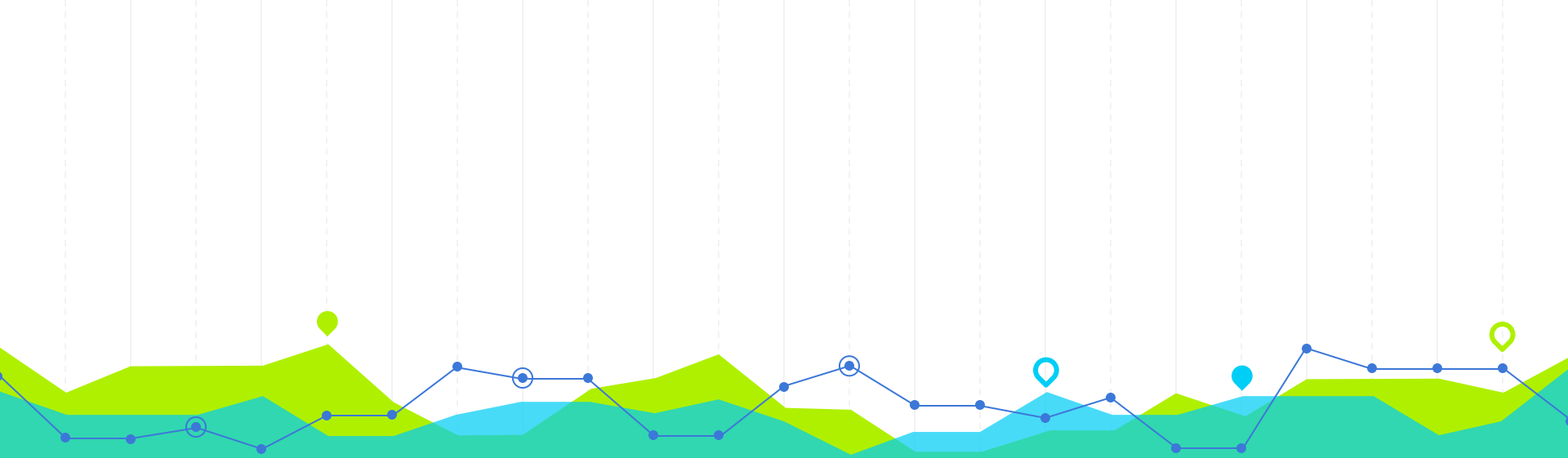
# My Skill Name

[Add your instructions here that Claude will follow when this skill is active]

## Examples
- Example usage 1
- Example usage 2

## Guidelines
- Guideline 1
- Guideline 2
```

*Exemple de fichier décrivant un skill
pour Claude Code*



1. Structure d'un projet Python

Pourquoi structurer un projet ?



Une structure claire facilite...

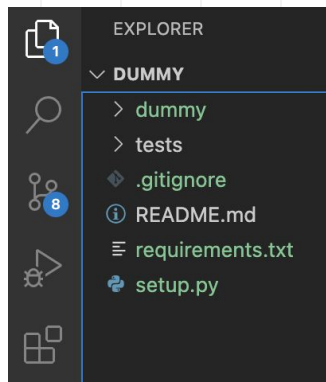
- **la lecture** et la **compréhension** du projet
- l'ajout de nouvelles fonctionnalités ou **l'amélioration / la scalabilité** des fonctionnalités existantes
- le **maintien** et le **débogue**

Alors que des projets mal structurés rend difficile...

- **la navigation** dans certaines fonctionnalités ou la **compréhension** de comment les différents composants interagissent.
- **la non-duplication de code**, ce qui conduit à des bogues et augmente le coût de la maintenance.
- l'organisation des **tests** ou peut même décourager la rédaction de tests



Structure globale d'un projet Python



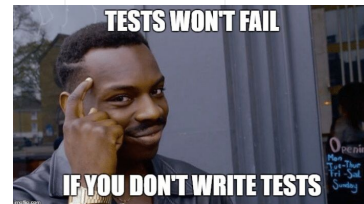
Structure de base

- `README.md`: Fichier pour présenter le projet, expliquer comment l'installer, l'utiliser, etc.
- `requirements.txt` ou `Pipfile` ou `environment.yaml` (conda), `pyproject.toml` : liste des dépendances
- `.git`: dossier de Git contenant l'historique et les métadonnées
- `.gitignore`: Liste des fichiers/dossiers que Git doit ignorer.
- `setup.py`: pour packager votre projet
- `Dockerfile` et `docker-compose.yml`: si le projet doit être déployé dans des conteneurs Docker

Dossiers du code source

- `src/` ou le nom de votre projet : Contient le code source principal.
 - `__init__.py`: Fichier qui marque le répertoire comme un package ou un module Python.
 - `module1.py`, `module2.py`: Fichiers Python contenant les classes, fonctions, etc.

Structure globale d'un projet Python



Scripts et exécutables

`bin/` ou `scripts/`: Scripts exécutables, programmes de démarrage, etc. Ils utilisent souvent le code défini dans `src/`.

Organisation des tests

`tests/`: Dossier contenant tous les tests.

- `__init__.py`: comme précédemment, marque le répertoire comme un package/module Python.
- `test_module1.py`, `test_module2.py`: fichiers contenant les tests unitaires et d'intégration pour les modules correspondants.
- `conftest.py`: configuration commune pour `pytest`, si utilisé.
- `fixtures/`: pour les données de test ou les objets mock.

Autres dossiers communs

- `docs/`: documentation du projet, souvent avec Sphinx.
- `data/`: données utilisées, comme des fichiers CSV, des configurations, etc.
- `config/`: fichiers de configuration spécifiques, comme des configurations de base de données.
- `examples/`: exemples d'applications utilisant Flask

Exemples de structures de projets



```

mon_projet/
├── .gitignore
├── README.md
├── docs/
│   └── index.md
├── ...
├── tests/
│   ├── __init__.py
│   └── test_module1.py
├── ...
├── mon_projet/
│   ├── __init__.py
│   ├── module1.py
│   ├── module2.py
│   ├── ...
│   └── utils/
│       ├── __init__.py
│       └── helper1.py
├── ...
├── data/
│   ├── processed/
│   └── raw/
├── scripts/
│   └── script1.py
├── ...
├── setup.py
├── requirements.txt
├── environment.yml # si vous utilisez Conda
├── Dockerfile # si le projet est déployé dans un conteneur
└── docker-compose.yml
    
```

```

mon_projet/
├── .git/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   └── cd.yml
│   └── .vscode/
│       ├── settings.json
│       └── tasks.json
├── docs/
│   ├── _build/
│   ├── _static/
│   ├── templates/
│   └── conf.py
├── nom_du_projet/
│   ├── __init__.py
│   ├── module1.py
│   └── module2.py
├── ...
├── utils/
│   ├── __init__.py
│   └── logger.py
├── ...
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── ...
├── venv/
├── .gitignore
├── .readthedocs.yml
├── README.md
├── requirements.txt
└── setup.py
    
```

Configuration et source pour GIT

Configuration CI/CD pour GitHub Actions

Configuration spécifique pour Visual Studio Code

Documentation du projet

Configuration pour Sphinx

Code source principal du projet

Outils et modules utilitaires

Configuration du logger

Tests unitaires

Environnement virtuel Python

Liste des fichiers/dossiers ignorés par GIT

Configuration pour l'hébergement de la doc sur ReadTheDocs

Description du projet

Dépendances du projet

Script de setup pour packaging et distribution

```

mon_projet/
├── .git/
├── .gitignore
├── .github/
│   ├── workflows/
│   │   └── ci.yml
├── docs/
│   ├── make.bat
│   ├── Makefile
│   ├── source/
│   │   ├── conf.py
│   │   └── index.rst
│   ├── ...
│   └── build/
├── mon_projet/
│   ├── __init__.py
│   ├── module1.py
│   └── module2.py
├── ...
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── ...
├── venv/
├── requirements.txt
├── setup.py
└── README.md
    
```

Répertoire Git pour le contrôle de version

Fichier pour ignorer certains fichiers/dossiers dans Git

Fichiers de configuration pour GitHub Actions (CI/CD)

CI/CD pour tests unitaires, build, etc.

Documentation du projet

Script pour générer la documentation avec Sphinx (Windows)

Script pour générer la documentation avec Sphinx (Unix)

Configuration Sphinx

Page d'accueil de la documentation

Autres fichiers .rst pour la documentation

Où Sphinx stocke les fichiers générés (e.g., HTML, LaTeX)

Fichier d'initialisation du package

Un module du projet

Un autre module du projet

Fichier d'initialisation pour rendre ce dossier un package

Tests pour module1

Tests pour module2

Environnement virtuel (optionnel)

Dépendances du projet

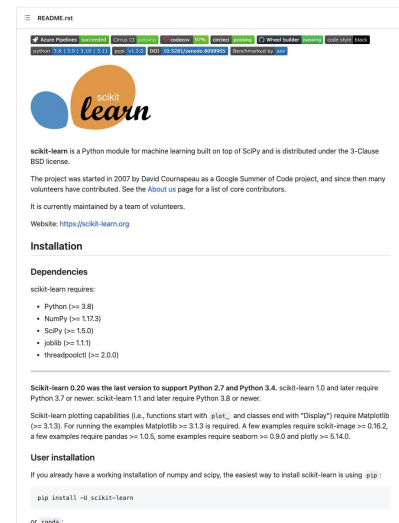
Fichier pour rendre le projet installable via pip

Présentation du projet, comment l'installer, l'utiliser, etc.

README.md

Le fichier README.md

- **Définition** : Un document texte qui présente et explique le projet
- **Importance** :
 - Première chose que d'autres développeurs (ou vous-même dans le futur) voient
 - Fournit des instructions pour l'**installation**, l'**utilisation** et la **contribution**
- **Contenu typique** :
 - Description du projet.
 - Comment installer et utiliser le projet.
 - Informations sur la licence, les auteurs, etc.
 - Badges pour la CI, la couverture des tests, etc. (si applicable).





`__init__.py`

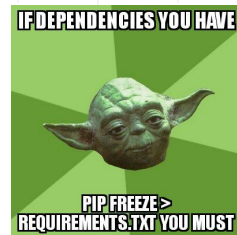
Le fichier `__init__.py`

Ce fichier est exécuté **chaque fois** que le package (ou sous-package) est importé.

Vous pouvez l'utiliser pour :

- **initialiser** des variables
- **importer** des modules
- définir des comportements spécifiques lors de l'initialisation du package
- rendre certaines fonctions **facilement accessibles** dès l'importation du package
- spécifier la version de votre package (cf `setup.py`)
- etc.

requirements.txt



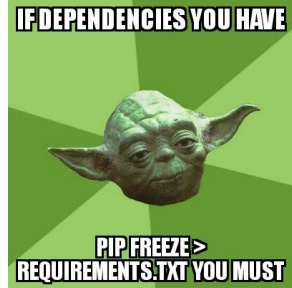
La gestion des dépendances et des requirements assure la **stabilité**, la **reproductibilité** et la **sécurité** des projets Python

Les dépendances avec **requirements.txt** ou **Pipfile**

- **Définition** : fichiers qui listent toutes les bibliothèques externes nécessaires
- **Pourquoi c'est crucial**
 - Assure la reproductibilité : **n'importe qui peut installer** les bonnes versions des dépendances.
 - Éviter les "ça marche sur ma machine" et les problèmes liés aux versions.
- **Format**
 - **requirements.txt** : liste plate, souvent générée avec `pip freeze > requirements.txt`.
 - **Pipfile** : utilisé avec **Pipenv**, permet une meilleure gestion et verrouillage des dépendances.
- **Application** :
 - `pip install -r requirements.txt`

```
requirements.txt U X
requirements.txt
1 networkx==3.1
2 voila==0.4.0
3 matplotlib==3.7.1
4
```

La gestion des dépendances : Les requirements



Bonnes pratiques pour gérer les **requirements** en Python

1. **Spécifiez les versions**
 - Utilisez des versions explicites pour garantir la **reproductibilité** (`flask==1.1.2` plutôt que `flask`).
 - utiliser des opérateurs de comparaison pour autoriser une **gamme de versions** (ex : `flask>=1.1, <2.0`).
2. **Divisez vos requirements**
 - Diviser votre `requirements.txt` en plusieurs fichiers (`base.txt`, `dev.txt`, `test.txt`, etc.).
3. **Mettez à jour régulièrement**
 - Utilisez des outils comme `pip-tools` ou `Dependabot` pour être informé des **mise à jour** de vos dépendances
4. **Testez après les mises à jour**
 - Lorsque vous mettez à jour une dépendance, **tester** votre application pour éviter toute **régression**
5. **Gérez les conflits**
 - Si deux dépendances dépendent de la même sous-dépendance mais de versions différentes, risque de **conflit**.
6. **Considérez l'utilisation d'un outil de verrouillage**
 - `pipenv` ou `poetry` offrent un mécanisme de verrouillage qui capture toutes les versions des dépendances
7. **Faites attention aux dépendances abandonnées**
 - elles pourraient avoir des vulnérabilités non corrigées ou ne plus être compatibles

La gestion des dépendances :

== ou >=



Fixer les versions (==)

Avantages

1. **Reproductibilité** : l'environnement sera **identique** à chaque installation
2. **Stabilité** : vous évitez d'introduire involontairement des **bugs** lorsqu'une nouvelle version d'une dépendance est publiée et qu'elle n'est pas totalement **compatible** avec votre code
3. **Sécurité** : vous pouvez **surveiller** ces versions spécifiques pour les problèmes de sécurité

Inconvénients

1. **Maintenance** : vous devrez **régulièrement mettre à jour manuellement** les versions pour bénéficier des corrections de bugs, des nouvelles fonctionnalités ou des mises à jour de sécurité
2. **Compatibilité** : si votre projet est une bibliothèque destinée à être utilisée par d'autres, fixer les versions peut causer des **conflits de dépendances** pour vos utilisateurs

La gestion des dépendances :

== ou >=



Spécifier une version minimale (>=)

Avantages

1. **Actualité** : votre projet bénéficiera toujours des **dernières versions des dépendances**
2. **Facilité de maintenance** : vous n'aurez pas à mettre à jour manuellement chaque dépendance aussi souvent

Inconvénients

1. **Stabilité** : une nouvelle version d'une dépendance peut introduire des changements qui cassent la compatibilité
2. **Reproductibilité** : différentes installations de votre projet à différents moments = des environnements différents

Recommandation

- **Pour les applications et les projets finaux** : Il est généralement recommandé de **fixer** les versions pour assurer la **stabilité** et la **reproductibilité** (**pip-tools**, **pipenv** ou **poetry** pour vous aider à mettre à jour vos dépendances)
- **Pour les bibliothèques** : destinée à être utilisée dans d'autres projets, il est souvent préférable d'utiliser **>=** (avec une spécification de version maximale si nécessaire) pour éviter des **conflits de dépendances trop restrictifs** pour vos utilisateurs
=> utiliser un service d'**intégration continue (CI)** qui teste régulièrement votre projet avec les dernières dépendances

Poetry



Poetry

Gestion des dépendances et verrouillage

Unifiée dans un seul fichier `pyproject.toml` et génère un fichier `poetry.lock` pour garantir des installations reproductibles.

Verrouillage de dépendances

Fichier `poetry.lock` pour des installations reproductibles.

Gestion des environnements virtuels

Crée et gère automatiquement les environnements virtuels à chaque projet. Permet de séparer facilement les dépendances nécessaires pour le dev de la production (black, ruff, etc.)

Résolution des conflits de dépendances

Meilleure résolution des versions pour éviter les conflits.

Publication sur PyPI

Intégrée pour construire et publier directement des packages sur PyPI ou d'autres dépôts.

Gestion des scripts

Permet de définir et exécuter des scripts personnalisés dans `pyproject.toml`.

Mise à jour sélective des dépendances

Permet de mettre à jour une dépendance et ses sous-dépendances sans affecter les autres.

Dépendances privées

Gère facilement les dépendances privées ou celles hébergées sur d'autres dépôts que PyPI

Pip

Utilise `requirements.txt`, mais sans fichier de verrouillage dédié (peut nécessiter `pip-tools` pour verrouillage).

Verrouillage possible via `pip freeze`, mais manuel et sans garantie de reproductibilité. (il existe la lib `pipreqs` pour générer un `requirements.txt` qui analyse le code pour identifier uniquement les dépendances réellement nécessaires à votre projet)

Requiert un outil séparé comme `virtualenv` ou `venv`.

Peut rencontrer des problèmes de conflits de versions.

Nécessite des outils supplémentaires comme `twine` pour publier.

Pas de fonctionnalité native pour la gestion des scripts.

`pip install --upgrade` met à jour toutes les dépendances.

Possible, mais nécessite une configuration supplémentaire.

Poetry



Poetry est un outil de gestion des dépendances qui vise à rendre ces processus plus intuitifs, flexibles et fiables par rapport aux outils existants comme `pip`, `setuptools` et `pipenv`.

Installation de Poetry

```
curl -sSL https://install.python-poetry.org | python3 -
```

Initialiser un nouveau projet

Si vous démarrez un nouveau projet, utilisez :

```
poetry new [mon-projet]
```

Ajouter Poetry à un projet existant

Accédez au répertoire de votre projet et utilisez :

```
poetry init
```

Ajouter des dépendances

```
poetry add [nom-du-paquet]
```

Pour les dépendances de développement (par exemple, des linters ou des outils de test) :

```
poetry add [nom-du-paquet] --dev
```

Installer les dépendances

Une fois que vous avez ajouté toutes vos dépendances, utilisez :

```
poetry install
```

Mise à jour des dépendances

Pour mettre à jour une dépendance spécifique :

```
poetry update [nom-du-paquet]
```

Pour mettre à jour toutes les dépendances :

```
poetry update
```

Poetry



Gestion des environnements virtuels

Poetry crée automatiquement un environnement virtuel pour chaque projet. Pour activer l'environnement virtuel :

```
poetry shell
```

Construire et publier

Pour construire un paquet distribuable :

```
poetry build
```

Pour publier votre paquet sur PyPI :

```
poetry publish --build
```

Gérer les scripts : dans le fichier `pyproject.toml`, vous pouvez ajouter des scripts pour faciliter certaines commandes :

```
[tool.poetry.scripts]
```

```
test = "pytest"
```

Vous pouvez ensuite lancer `pytest` avec la commande :

```
poetry run test
```

Lister les dépendances

```
poetry show
```

Supprimer des dépendances

```
poetry remove [nom-du-paquet]
```

Gérer plusieurs versions de Python

Spécifier la version de Python dans `pyproject.toml` et utiliser `pyenv` pour gérer plusieurs versions de Python sur votre machine.

Utiliser le verrouillage

Le fichier `poetry.lock` garantit que tous ceux qui travaillent sur le projet utilisent les mêmes dépendances. Committez-le.

Configurer des sources alternatives

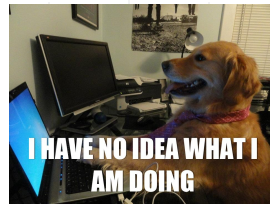
Si vous avez besoin de paquets à partir de référentiels autres que PyPI, vous pouvez les configurer dans `pyproject.toml`

Exporter vers `requirements.txt`

```
poetry export -f requirements.txt > requirements.txt
```

setup.py => pyproject.toml

setup.py



décrit les informations essentielles sur un projet Python (nom, version, auteur, description, dépendances, etc.). Il est utilisé pour :

- **Installation** : installer votre projet sur votre système en utilisant la commande `python setup.py install`. Cela copiera les fichiers de votre projet dans l'environnement Python de l'utilisateur.

```
python setup.py install
```

- **Distribution** : créer des distributions de votre projet, généralement au format de package Python (souvent un fichier `.tar.gz` ou `.zip`). Ces distributions peuvent être partagées avec d'autres utilisateurs afin qu'ils puissent installer votre projet.

```
python setup.py sdist # créer une distribution source (.tar.gz).
```

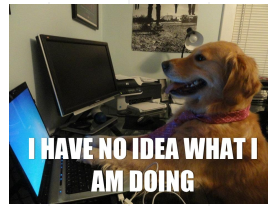
```
python setup.py bdist_wheel # créer une distribution binaire au format Wheel
```

```
python setup.py register # enregistrer votre projet sur le Python Package Index (PyPI)
```

- **Gestion des dépendances** : spécifier les dépendances requises pour votre projet dans `setup.py`

`setup.py => pyproject.toml`

setup.py



```
from setuptools import setup, find_packages
```

```
setup(  
    name="nom_du_projet",  
    version="0.1",  
    description="Description du projet",  
    author="Votre nom",  
    packages=find_packages(),  
    install_requires=[  
        # Liste des dépendances requises  
        "numpy",  
        "matplotlib",  
    ],  
)
```

Le fichier setup.py utilise le module setuptools, qui est un ensemble d'outils Python pour la distribution de packages.

- name : nom de votre projet.
- version : version de votre projet.
- description : description du projet.
- author : nom ou le nom de l'auteur du projet.
- packages : liste des packages inclus dans votre projet.
find_packages() peut être utilisé pour trouver automatiquement tous les packages dans votre projet.
- install_requires : liste des dépendances requises par votre projet.
Lorsque quelqu'un installe votre projet, setuptools s'assurera que ces dépendances sont également installées.

pyproject.toml

```
[build-system]
# Hatch s'appuie sur uv pour gérer les dépendances et la résolution rapide
requires = ["hatchling", "uv"]
build-backend = "hatchling.build"

[project]
name = "mon-projet"
version = "0.1.0"
description = "Un exemple de projet Python utilisant Hatch + uv"
authors = [
    { name = "Jean Dupont", email = "jean.dupont@example.com" }
]
license = "MIT"
readme = "README.md"
requires-python = ">=3.10"

# Dépendances principales du projet
dependencies = [
    "requests",
    "pydantic>=2.0",
]

[project.optional-dependencies]
# Dépendances supplémentaires installables avec : uv pip install .[dev]
dev = [
    "pytest",
    "black",
    "ruff",
]
```

```
[tool.hatch.envs.default]
# Configuration de l'environnement "par défaut" avec uv comme pip backend
type = "virtual"
pip-implementation = "uv" # 🔄 Indique à Hatch d'utiliser uv au lieu de pip

[tool.hatch.envs.dev]
# Environnement spécifique pour le développement
inherit = "default"
dependencies = [
    "pytest",
    "black",
    "ruff",
]

[tool.hatch.metadata.hooks.uv]
# Permet à Hatch de déléguer la résolution de dépendances à uv
```

pyproject.toml

```
from setuptools import setup, find_packages

setup(
    name="mon-projet",
    version="0.1.0",
    description="Un projet exemple avec setup.py",
    author="Jean Dupont",
    author_email="jean.dupont@example.com",
    license="MIT",
    packages=find_packages(),
    python_requires=">=3.10",
    install_requires=[
        "requests",
        "pydantic>=2.0",
    ],
    extras_require={
        "dev": [
            "pytest",
            "black",
            "ruff",
        ]
    },
)
```

setup.py => pyproject.toml

```
[build-system]
requires = ["hatchling", "uv"]
build-backend = "hatchling.build"

[project]
name = "mon-projet"
version = "0.1.0"
description = "Un projet exemple migré vers pyproject.toml"
authors = [
    { name = "Jean Dupont", email = "jean.dupont@example.com" }
]
license = "MIT"
readme = "README.md"
requires-python = ">=3.10"

dependencies = [
    "requests",
    "pydantic>=2.0",
]

[project.optional-dependencies]
dev = [
    "pytest",
    "black",
    "ruff",
]

[tool.hatch.envs.default]
type = "virtual"
pip-implementation = "uv"

[tool.hatch.envs.dev]
inherit = "default"
dependencies = [
    "pytest",
    "black",
    "ruff",
]
```

Lint et formateur

pylint → gros inspecteur de qualité (bug + style + structure)

ruff → couteau suisse moderne (rapide, linter + formateur)

black → reformateur strict, sans compromis

Flake8 → vérif style basique, extensible par plugins

autopep8 → formateur simple (PEP8 only), plus ancien que Black

Outil	Type principal	Fonctionnalité	Points forts	Limites
Pylint	Lint complet	Vérifie les erreurs de code, la qualité, la complexité, les conventions de nommage, la duplication, etc.	Très complet, configurable, peut détecter des bugs logiques	Lent, parfois trop strict/verbeux
Ruff	Lint + formateur moderne (remplace Flake8, isort, pydocstyle, pyupgrade, etc.)	Ultra rapide (Rust), vérifie style + erreurs, peut corriger automatiquement	Extrêmement rapide, remplace plusieurs outils en un seul	Moins configurable que Pylint pour les règles avancées
Black	Formateur d'opinion	Reformate automatiquement le code Python selon une convention stricte	Zéro configuration, cohérent, adopté massivement	Pas flexible (style imposé)
Flake8	Lint léger	Vérifie le style PEP8 + quelques erreurs de base	Simple, extensible avec plugins	Moins complet que Pylint, plus lent que Ruff
autopep8	Formateur	Reformate le code pour respecter PEP8	Léger, simple	Moins strict que Black, pas aussi moderne

Mais aussi Bandit, pyflakes, YAPF, isort, etc.

.gitignore



Ignorer des fichiers avec **.gitignore**

- **Définition** : un fichier pour dire à Git quels fichiers ou dossiers ne doivent pas être suivis
- **Pourquoi c'est important** :
 - certains fichiers n'ont pas leur place dans un dépôt (fichiers temporaires, configurations locales, etc.)
 - protège contre l'ajout accidentel de fichiers sensibles (mots de passe, clés API)
- **Exemples courants à ignorer** :
 - Dossiers `__pycache__`, fichiers `.pyc`.
 - Environnements virtuels
 - Fichiers de configuration personnels (`.env`)

Scripts et exécutables scripts/



Placement et organisation

- Dossier `bin/` ou `scripts/` : où placer les scripts dans la structure du projet
- Donner un nom descriptif : le nom du script doit refléter sa fonction (e.g., `backup_database.py`, `start_server.py`).

Objectif des scripts

- **Automatiser** des tâches répétitives
- Servir de points d'entrée pour exécuter des **fonctionnalités spécifiques** d'un projet
- Exposer des commandes **CLI** (Command Line Interface) pour les utilisateurs ou d'autres services
- Shebang (`#!/usr/bin/env python3`) : pour permettre l'**exécution directe** sur des systèmes Unix

```
if __name__ == "__main__":  
    main()
```

Ceci garantit que le script peut être exécuté comme un script standalone et peut aussi être importé comme un module

Scripts et exécutables scripts/



Comment rendre un script exécutable

- Pour les systèmes Unix, utiliser la commande `chmod +x script_name.py` pour rendre le script exécutable
- Appel direct par `./script_name.py` après avoir ajouté le **shebang**
- Pour les systèmes Windows, les scripts Python sont généralement exécutés via `python script_name.py`.

Création d'un CLI avec `argparse`

- `argparse` : un module de la bibliothèque standard pour écrire des interfaces utilisateur en ligne de commande
- Définition des arguments, des options, des descriptions

Bonnes pratiques

- Éviter de mettre trop de logique dans un script, le **code complexe doit être dans des modules** appelés par le script
- Les scripts doivent **gérer correctement les erreurs** pour éviter de crasher de manière inattendue
- Chaque script doit avoir une **documentation** appropriée, en particulier s'il est destiné à être utilisé par d'autres

Dossier du code source src/

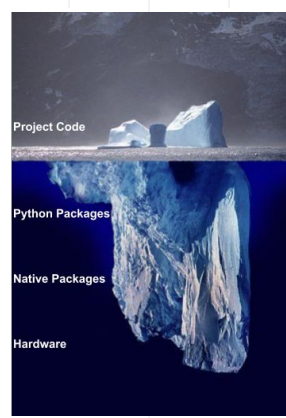
Pourquoi organiser le code dans un dossier spécifique

- **séparation du code** source des autres fichiers non liés au code (README, licences, scripts)
- **facilite l'importation** de modules entre eux
- rend clair **où chercher** les sources principales du projet

Dossier **src/** (ou nom du projet) comme conteneur du code source

Organiser les classes et les fonctions

- répartition des classes et fonctions dans des fichiers distincts pour une meilleure lisibilité
- **nommer** les fichiers de manière à refléter leur contenu





Dossier du code source : Les packages et comment les organiser

Les **packages** en Python sont une façon d'organiser les **modules** (fichiers `.py`) en **groupes hiérarchisés**

Les packages sont simplement des répertoires qui contiennent un fichier spécial nommé `__init__.py` (qui peut être vide)

=> facilite la **modularité** et la **factorisation** du code

Package `monpackage` contenant `module1` et `module2`.

```
monpackage/  
|  
├── __init__.py  
|  
├── module1.py  
|  
└── module2.py
```

Pour utiliser un module à partir de ce package :

```
from monpackage import module1
```

ou pour une fonction spécifique :

```
from monpackage.module1 import ma_fonction
```

Dossier du code source : Les packages et comment les organiser



monpackage/

├── __init__.py

├── module1.py

├── module2.py

└── souspackage/

 ├── __init__.py

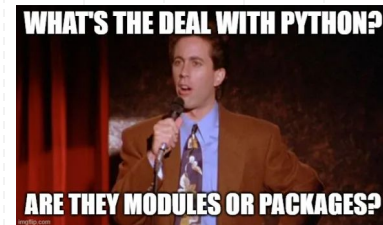
 ├── submodule1.py

 └── submodule2.py

Vous pouvez alors accéder aux sous-modules comme ceci :

```
from monpackage.souspackage import submodule1
```

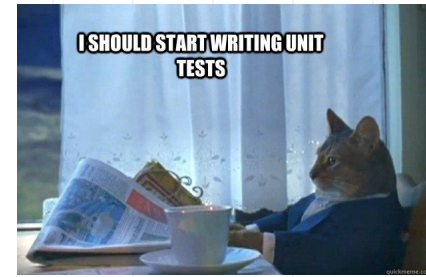
Dossier du code source : Les packages et comment les organiser



Conseils pour organiser les packages :

- **Modularité** : chaque module doit avoir une **fonction spécifique**. Décomposer les modules trop gros.
- Les **noms** des modules et des packages doivent être clairs et **réfléter leur utilité**.
- Minimiser les **dépendances** entre les modules. Cela facilite la **réutilisation** et le **test** des modules.
- Chaque package, module, classe et fonction doit être correctement **documenté**. Utilisez les **docstrings** (par exemple, grâce aux extensions VSCode 😎)

Organisation des tests

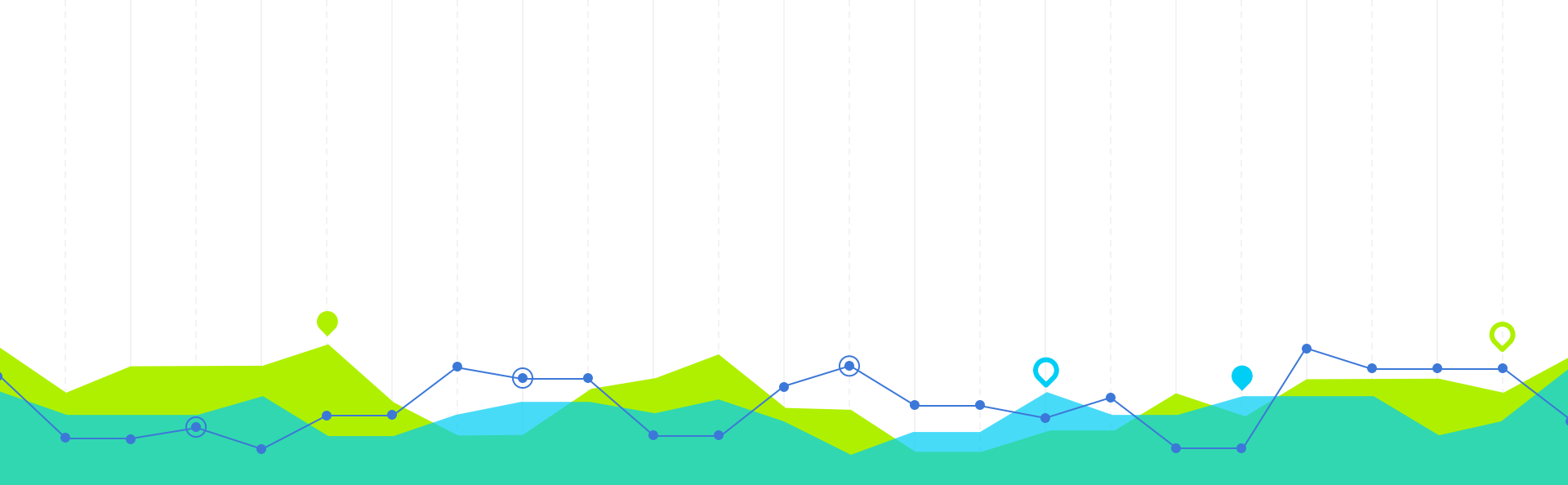


Dossier de tests

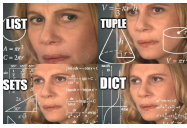
- `tests/` : dossier qui contient tous les tests

Structuration des fichiers de tests

- `test_module1.py`, `test_module2.py`: convention de nommage pour pytest / unittest des fichiers de tests
- un fichier de tests pour chaque module du code source.

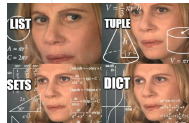


2. Programmation orientée objet en Python



Utilisation efficace des collections et structures de données

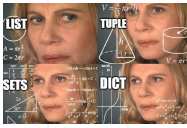
	Utilisation	Quand l'utiliser	Conseil	Exemple
Listes	Les listes sont utilisées pour stocker une collection ordonnée d'éléments.	Utilisez une liste lorsque l'ordre des éléments est important.	Évitez d'ajouter ou de supprimer des éléments au début d'une liste car cela nécessite de décaler tous les autres éléments. Utilisez <code>append()</code> et <code>pop()</code> pour ajouter/supprimer à la fin, car ils sont plus rapides.	<pre>my_list = [1, 2, 3, 4, 5] my_list.append(6) # Ajoute 6 à la fin de la liste print(my_list) # Affiche [1, 2, 3, 4, 5, 6]</pre>
Tuples	Similaires aux listes, mais immuables.	Lorsque vous avez besoin d'une liste qui ne change pas.	Les tuples peuvent être utilisés comme clés dans un dictionnaire, contrairement aux listes.	<pre>my_tuple = (1, 2, 3, 4, 5) print(my_tuple[1]) # Affiche 2</pre>
Ensembles (set)	Pour stocker une collection non ordonnée d'éléments uniques.	Lorsque vous voulez éviter les doublons ou effectuer des opérations ensemblistes (union, intersection).	Les vérifications d'appartenance (<code>in</code>) sont généralement plus rapides avec un ensemble qu'avec une liste.	<pre>my_set = {1, 2, 3, 4, 5} my_set.add(6) # Ajoute 6 à l'ensemble print(3 in my_set) # Affiche True, car 3 est dans l'ensemble</pre>



Utilisation efficace des collections et structures de données

	Utilisation	Quand l'utiliser	Conseil	Exemple
Dictionnaires (dict)	Pour stocker des paires clé-valeur.	Lorsque vous avez besoin d'une recherche rapide par clé.	Les clés d'un dictionnaire doivent être immuables. Si vous avez besoin de clés composées, vous pouvez utiliser un tuple.	<pre>my_dict = {"apple": 1, "banana": 2, "cherry": 3} my_dict["date"] = 4 # Ajoute une nouvelle paire clé-valeur print(my_dict.get("apple")) # Affiche 1</pre>
Deque (collections.deque)	C'est comme une liste, mais les ajouts et suppressions aux deux extrémités sont optimisés.	Idéal pour les files d'attente et les piles où vous devez ajouter/supprimer des éléments des deux côtés.	Utilisez <code>appendleft()</code> et <code>popleft()</code> pour ajouter/supprimer rapidement au début.	<pre>from collections import deque my_deque = deque([1, 2, 3, 4, 5]) my_deque.appendleft(0) # Ajoute 0 au début print(my_deque.pop()) # Affiche 5 et le retire de la fin</pre>
Counter (collections.Counter)	Pour compter les occurrences d'éléments dans une collection.		Très utile pour analyser rapidement la fréquence des éléments dans une collection.	<pre>from collections import Counter my_counter = Counter(["apple", "banana", "apple", "cherry", "banana", "apple"]) print(my_counter) # Affiche Counter({'apple': 3, 'banana': 2, 'cherry': 1})</pre>

Utilisation efficace des collections et structures de données



	Utilisation	Quand l'utiliser	Conseil	Exemple
DefaultDict (collection s.defaultdict ct)	Comme un dictionnaire, mais avec une valeur par défaut pour les clés manquantes.		Utile lorsque vous voulez éviter de vérifier constamment si une clé est déjà présente dans le dictionnaire.	<pre>from collections import defaultdict my_default_dict = defaultdict(int) my_default_dict["apple"] += 1 print(my_default_dict["banana"]) # Affiche 0, car la valeur par défaut est int (0)</pre>
Heaps (heapq)	Pour les structures de données de type tas.		Lorsque vous avez besoin d'une file de priorité, par exemple pour trouver rapidement les n plus petits ou n plus grands éléments.	<pre>import heapq my_heap = [4, 7, 3, 1, 8] heapq.heapify(my_heap) # Transforme la liste en tas print(heapq.heappop(my_heap)) # Affiche 1, le plus petit élément</pre>



Lambda, map, filter, reduce

Fonction	Description	Code d'exemple
lambda	Fonction anonyme généralement utilisée pour des opérations simples	<pre># Définition d'une fonction lambda pour calculer le carré d'un nombre carre = lambda x: x**2 # Utilisation de la fonction lambda resultat = carre(5) print(resultat) # Affiche 25</pre>
map	La fonction map applique une fonction à chaque élément d'une séquence (comme une liste) et renvoie un objet map. Vous pouvez le convertir en liste ou en autre type si nécessaire	<pre># Liste de nombres nombres = [1, 2, 3, 4, 5] # Utilisation de map pour doubler chaque élément de la liste doubles = map(lambda x: x * 2, nombres) # Conversion de l'objet map en liste doubles_liste = list(doubles) print(doubles_liste) # Affiche [2, 4, 6, 8, 10]</pre>



Lambda, map, filter, reduce

Fonction	Description	Code d'exemple
filter	La fonction filter filtre les éléments d'une séquence en fonction d'une fonction de test (lambda ou autre). Elle renvoie un objet filter, que vous pouvez également convertir en une autre séquence.	<pre># Liste de nombres nombres = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] # Utilisation de filter pour ne garder que les nombres pairs nombres_pairs = filter(lambda x: x % 2 == 0, nombres) # Conversion de l'objet filter en liste nombres_pairs_liste = list(nombres_pairs) print(nombres_pairs_liste) # Affiche [2, 4, 6, 8, 10]</pre>
reduce	La fonction reduce (qui se trouve dans le module functools) applique une fonction cumulative à une séquence et renvoie un seul résultat.	<pre>from functools import reduce # Liste de nombres nombres = [1, 2, 3, 4, 5] # Utilisation de reduce pour calculer la somme des nombres somme = reduce(lambda x, y: x + y, nombres) print(somme) # Affiche 15 (1 + 2 + 3 + 4 + 5)</pre>

Le typage

Le typage statique est devenu de plus en plus populaire avec l'introduction des annotations de type dans Python 3.5 (PEP 484).

Utilisez des annotations de type pour documenter le type attendu des arguments et la valeur de retour d'une fonction.

```
def greet(name: str) -> str:  
    return "Hello, " + name
```

Utilisez **typing** pour les types complexes

```
from typing import List, Dict  
def get_names(people: List[Dict[str, str]]) -> List[str]:  
    return [person["name"] for person in people]
```



Le typage



Utilisez Optional pour les valeurs qui peuvent être None

```
from typing import Optional

def find_user(user_id: int) -> Optional[Dict[str, str]]:
    # Si l'utilisateur n'est pas trouvé, retourne None
    # Sinon, retourne le dictionnaire de l'utilisateur
    ...
```

Typez vos classes et méthodes

```
class User:

    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age

    def display(self) -> str:
        return f"{self.name}, {self.age} years old"
```

Le typage

Utilisez Union pour les valeurs qui peuvent avoir plusieurs types

```
from typing import Union
def add(a: Union[int, float], b: Union[int, float]) -> Union[int, float]:
    return a + b
```

Annotez les variables lorsque cela est nécessaire

```
from typing import List
names: List[str] = ["Alice", "Bob", "Charlie"]
```

Évitez **Any** sauf si absolument nécessaire

Son utilisation signifie que vous renoncez à la vérification de type pour cette partie du code.

```
from typing import Any
def process(data: Any) -> None:
    ...
```



Le typage



Utilisez des classes de données (Python 3.7+) pour une typage plus propre et une meilleure lisibilité

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float = 4
```

Utilisez des outils de vérification de type

mypy, **pyright** et **pytype** sont quelques-uns des vérificateurs de type populaires qui peuvent être utilisés pour vérifier votre code pour les erreurs de typage.

Soyez cohérent

Si vous décidez d'utiliser le typage statique dans votre projet, essayez de l'adopter de manière cohérente à travers votre codebase



Pour aller plus loin : Traitlets

Bibliothèque qui facilite la gestion des attributs et des paramètres d'objet via un **mécanisme de déclaration** et de **validation**.

Traitlets permet de **définir des types et des valeurs par défaut** pour les **attributs**, de **valider les entrées utilisateur**, de **surveiller les modifications** d'attributs, et offre une syntaxe concise pour gérer ces opérations.

Quand l'utiliser ?

Lorsque vous avez besoin de créer des objets avec des attributs configurables de manière flexible et que vous souhaitez garantir la cohérence des données.

```
from traitlets import HasTraits, Int, validate, TraitError

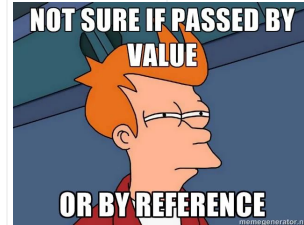
class TemperatureSensor(HasTraits):
    temperature = Int(default_value=25, min=0, max=100)

# Créer une instance de TemperatureSensor
sensor = TemperatureSensor()

# Définir une température valide
sensor.temperature = 30
print(sensor.temperature) # Affiche 30

# Essayons de définir une température invalide
try:
    sensor.temperature = -5
except TraitError as e:
    print(e) # Affiche "The value of the 'temperature' trait of a TemperatureSensor
            # instance should not be less than 0, but a value of -5 was specified"
```

Passage par valeur ou par référence ?



Par valeur

```
def modify_value(x):  
    x += 10  
  
num = 5  
modify_value(num)  
print(num) # num reste égal à 5, car int est immuable
```

Par référence

```
def modify_list(lst):  
    lst.append(4)  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(my_list) # my_list contient maintenant [1, 2, 3, 4]
```

```
def modify_list(lst):  
    lst = lst.copy() # nouvelle référence de l'objet  
    lst.append(4)  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(my_list) # my_list reste [1, 2, 3]
```

```
def modify_list(lst):  
    lst = [999] # nouvelle référence de l'objet  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(my_list) # my_list reste [1, 2, 3]
```

le passage **par valeur** s'applique sur les types primitifs / immuables,
le passage **par référence** s'applique sur les types composés / mutable.

En clair, si dans une fonction :

- modification d'un objet **immuable**
= modification **non visible** en dehors de la fonction
- modification d'un objet **mutable**
= modifications **visibles** en dehors de la fonction
- **réassigner la référence** à un nouvel objet
= modification **non visible** en dehors de la fonction

P00 en Python

Paradigme de programmation qui repose sur la création de **classes** et d'**objets** :

Classe : un modèle ou un plan pour créer des **objets**. Elle définit les **attributs** (variables) et les **méthodes** (fonctions) que les objets auront.

Objet : une **instance** d'une classe. Il est **créé à partir de la classe** et possède **ses propres attributs et méthodes**.



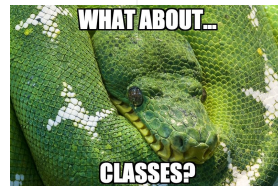
```
class Voiture:
```

```
    def __init__(self):  
        self.nom = "Ferrari"
```

```
    def donne_moi_le_modele(self):  
        return "250"
```

```
# crée un objet  
ma_voiture = Voiture()  
  
# appelle une méthode  
ma_voiture.donne_moi_le_modele()# affiche '250'  
  
# aperçu des méthodes de l'objet :  
dir(ma_voiture) # affiche ['__class__', '__delattr__', '__dict__', ..., 'donne_moi_le_modele', 'nom']  
  
# attribut spécial vous donne les valeurs des attributs de l'instance  
ma_voiture.__dict__ # affiche {'nom': 'Ferrari'}
```

Exemples de classes



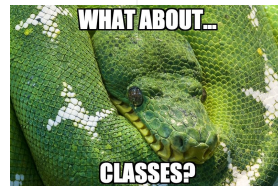
- **voiture** pour représenter des voitures avec des attributs tels que la marque, le modèle, la couleur, etc.
- **personne** pour représenter des individus avec des attributs tels que le nom, l'âge, l'adresse, etc.
- interfaces utilisateur graphiques (**GUI**) avec des éléments tels que des fenêtres, des boutons et des boîtes de dialogue.
- jeux vidéo : **personnages**, des **ennemis**, des **objets**, etc., avec leurs propriétés et comportements spécifiques.
- données : **structures** de données complexes, telles que des **arbres**, des **graphes** ou des **modèles** de ML
- réseaux : **sockets**, des **connexions réseau**, etc.
- frameworks web : **routes**, des **contrôleurs**, des **modèles** et d'autres **composants** d'une application web.
- classes pour contrôler des dispositifs matériels tels que des **capteurs**, des **actionneurs** et des **microcontrôleurs**.
- classes pour représenter des **comptes bancaires**, des **transactions**, des **portefeuilles**, etc.
- classes pour modéliser des **phénomènes physiques**, des **processus biologiques** ou des **systèmes complexes**.
- ... n'importe quelle **entité** !

Lorsque vous utilisez des bibliothèques tierces, vous interagissez souvent avec des **classes** fournies par ces bibliothèques :

```
from sklearn.cluster import KMeans
import numpy as np

X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])

# Création d'une instance de la classe KMeans
kmeans = KMeans(n_clusters=2, random_state=0, n_init= "auto")
# Utilisation de cette instance sur nos données
kmeans.fit(X)
kmeans.labels_
```



Comment créer une classe

1. **Nommer la classe** : Utilisez la convention `CamelCase` pour nommer les classes. Les noms doivent être descriptifs et clairs.
2. **Docstrings** : Utilisez les docstrings pour documenter la classe, ses méthodes et ses attributs.
3. **Constructeur** : Utilisez la méthode spéciale `__init__` pour initialiser les attributs de votre classe.
4. **Encapsulation** : Rendez les attributs privés (ou protégés) en les préfixant par un seul ou double underscore (`_` ou `__`). Si vous voulez permettre l'accès extérieur à ces attributs, utilisez des méthodes d'accès (getters) et de modification (setters).
5. **Méthodes** : Si une méthode ne doit pas accéder aux attributs ou méthodes d'instance, considérez à la rendre statique en utilisant le décorateur `@staticmethod`.
6. **Héritage** : Si vous souhaitez que votre classe hérite de la fonctionnalité d'une autre classe, faites-le clairement.
7. **Représentation de l'objet** : Pour une représentation lisible de votre objet, implémentez la méthode `__str__`. Pour une représentation formelle (souvent utilisée pour la débogage), implémentez `__repr__`.
8. **D'autres méthodes magiques** : plusieurs méthodes magiques, comme `__len__`, `__getitem__`, `__setitem__`, etc.
9. **Eviter le code redondant** : Si vous trouvez que certaines parties du code sont répétitives, envisagez de les refactorer.
10. **Testez votre classe** : Écrivez des tests unitaires pour vous assurer que votre classe fonctionne comme prévu.

Comment créer une classe

Les dataclasses, introduites en Python 3.7 via le module `dataclasses`, offrent une manière simplifiée de créer des classes, principalement pour stocker des données. Voici quelques avantages de leur utilisation :

1. **Moins de code boilerplate** : Avec les dataclasses, vous n'avez pas besoin d'écrire explicitement des méthodes comme `__init__`, `__repr__`, et `__eq__`. Elles sont générées automatiquement.
2. **Annotations de type** : Les dataclasses utilisent les annotations de type pour définir les attributs, ce qui rend le code plus lisible et permet d'utiliser des outils de vérification de type comme `mypy`.
3. **Valeurs par défaut** : Les dataclasses facilitent l'assignation de valeurs par défaut aux attributs.
4. **Immutabilité** : Avec l'option `frozen=True`, une dataclass peut être rendue immuable, ce qui signifie que ses attributs ne peuvent pas être modifiés après la création de l'objet.
5. **Utilitaires intégrés** : Le module `dataclasses` fournit plusieurs fonctions utilitaires comme `asdict` et `astuple` pour convertir l'objet dataclass en dictionnaire ou en tuple, respectivement.

Using dataclasses

Using
dataclasses
+ marshmallow

Using
dataclasses
+ marshmallow
+ desert



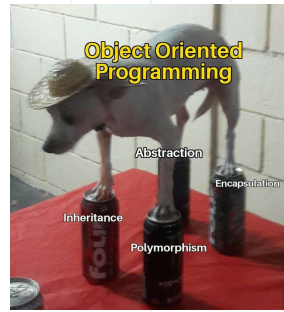
```
class Personne:
    def __init__(self, prenom, nom, age):
        self._prenom = prenom
        self._nom = nom
        self._age = age
```

```
from dataclasses import
dataclass

@dataclass
class Personne:
    prenom: str
    nom: str
    age: int

    def __post_init__(self):
        # constructeur
```

L'héritage



Permet de créer de **nouvelles classes** (sous-classes / classes filles) à **partir d'une classe existante** (classe mère), en héritant de ses attributs et méthodes. Cela favorise la réutilisation du code et la création de hiérarchies de classes.

```
class Voiture:

    roues = 4
    moteur = 1

    def __init__(self):
        self.nom = "A déterminer"

class VoitureSport(Voiture):

    def __init__(self):
        self.nom = "Ferrari"
```

```
ma_voiture = Voiture()
ma_voiture.nom # affiche 'A déterminer'
ma_voiture.roues # affiche 4

ma_voiture_sport = VoitureSport()
ma_voiture_sport.nom # affiche 'Ferrari'
ma_voiture_sport.roues # affiche 4
```

Le polymorphisme

```
class Voiture:

    roues = 4
    moteur = 1

    def __init__(self):
        self.nom = "A déterminer"

    def allumer(self):
        print( "La voiture démarre" )

class VoitureSport (Voiture):

    def __init__(self):
        self.nom = "Ferrari"

    def allumer(self):
        # on surcharge la méthode allumer de la classe mère
        Voiture.allumer(self) # ou super().allumer()
        print( "La voiture de sport démarre" )
```

permet à plusieurs classes d'implémenter des méthodes avec le même nom mais avec des comportements différents.
Cela facilite l'écriture de code générique et flexible.

```
ma_voiture_sport = VoitureSport()
ma_voiture_sport.allumer()
```

```
# affiche La voiture démarre
La voiture de sport démarre
```



```
from abc import ABC, abstractmethod# Abstract Base Classes
```

```
# classe abstraite  
class Voiture(ABC):
```

```
    roues = 4  
    moteur = 1
```

```
    def __init__(self):  
        self.nom = "A déterminer"
```

```
    @abstractmethod  
    def allumer(self):  
        print("La voiture démarre")
```

```
    def augmenter_vitesse(self, speed):  
        self.vitesse += speed
```

```
class VoitureSport(Voiture):
```

```
    def __init__(self, vitesse):  
        self.nom = "Ferrari"  
        self.vitesse = vitesse
```

```
    def allumer(self):  
        # on surcharge la méthode allumer de la classe mère  
        Voiture.allumer(self) # ou super().allumer()  
        print("La voiture de sport démarre")
```

```
    def compteur_vitesse(self):  
        print(f"Mon compteur affiche {self.vitesse}km")
```

L'abstraction

Supposons qu'on exige de ne jamais avoir des objets de la classe Voiture. Autrement dit, cette classe est créée pour devenir une mère de VoitureSport en imposant qu'elle ne sera jamais instanciée. Alors, on dit que cette classe doit être une **classe abstraite**.

```
ma_voiture = Voiture()# error : Can't instantiate abstract class Voiture with  
abstract method allumer
```

```
ma_voiture_sport = VoitureSport(20)
```

```
ma_voiture_sport.compteur_vitesse()# affiche Mon compteur affiche 20km
```

```
ma_voiture_sport.augmenter_vitesse(10)
```

```
ma_voiture_sport.compteur_vitesse()# affiche Mon compteur affiche 30km
```



Les décorateurs



Un décorateur est une fonction qui modifie le comportement d'autres fonctions.

Les décorateurs sont utiles lorsque l'on veut ajouter du même code à plusieurs fonctions existantes.

```
def smart_divide(func):
    def inner(a, b):
        print("I am going to divide", a, "and",
b)

        if b == 0:
            print("Whoops! cannot divide")
            return

        return func(a, b)
    return inner

@smart_divide
def divide(a, b):
    print(a/b)
```

```
divide(2,5)
# affiche :
# I am going to divide 2 and 5
# 0.4
```

```
divide(2,0)
# affiche :
# I am going to divide 2 and 0
# Whoops! cannot divide
```

Encapsulation



L'encapsulation est l'un des principaux concepts de la programmation orientée objet (POO).

Elle consiste à **restreindre l'accès** à certaines composantes d'un objet afin de prévenir des modifications non désirées.

Ceci est fait en masquant les détails internes d'un objet et en exposant seulement ce qui est nécessaire.

L'encapsulation aide à :

1. **Protéger** la modification accidentelle des données.
2. **Simplifier** l'utilisation d'objets.
3. Favoriser la **modularité** et la **flexibilité** du code.

Encapsulation



Usage de `_` et `__`:

- `_attr` : variable ou méthode **protégée**, ce qui signifie qu'elle ne doit pas être accédée directement, bien qu'elle le puisse techniquement (convention).
- `attr_` : variable **privée**, ce qui signifie que vous devriez la traiter comme si elle était destinée uniquement à être utilisée à l'intérieur de la classe ou de l'objet auquel elle appartient, bien qu'elle le puisse techniquement (convention).
- `__attr` : lorsque vous préfixez un attribut avec un double underscore, Python le renomme pour inclure un nom de classe spécifique. Ceci est appelé "name mangling". C'est une façon pour Python de **protéger** l'attribut et de prévenir les conflits de noms dans les sous-classes. Cela rend l'attribut moins accessible, mais pas totalement inaccessible.
- `__attr__` : variable spéciale ou "magique" associée à des méthodes spéciales de l'objet. Ces méthodes spéciales sont appelées méthodes de double underscore ou méthodes magiques (ex : `'__str__' -> str()`, `'__repr__' -> repr()`)

Encapsulation



Décorateurs de classe:

@staticmethod:

- Une méthode statique n'a pas accès à la classe elle-même ou aux instances de la classe.
- Elle n'a pas de paramètre `self` (instance) ou `cls` (classe) lors de sa définition.
- Elle est utilisée lorsqu'une méthode ne dépend pas des attributs d'instance ou de classe.
- Elle peut être appelée sur une classe plutôt que sur une instance.

```
class Math:
    @staticmethod
    def add(x, y):
        return x + y
```

Encapsulation



Public property

Private property
with public getter
and setter

@classmethod : méthode de classe

- a accès à la classe et non à l'instance.
- `cls` comme premier paramètre qui pointe vers la classe (et non l'instance).
- peut modifier le comportement de la classe, mais pas des instances spécifiques.
- Utilisée, par exemple, pour créer des constructeurs alternatifs

```
class Personne:
    population = 0

    def __init__(self, nom):
        self.nom = nom
        Personne.population += 1

    @classmethod
    def nombre_de_personnes(cls):
        return cls.population

Personne.nombre_de_personnes()# affiche 0
```

@property, @setter, @deleter:

- Utilisés pour définir les méthodes d'accès (getters), de modification (setters) et de suppression (deleters) pour un attribut.
- permet d'encapsuler l'accès à un attribut tout en ayant une syntaxe qui ressemble à un accès direct à un attribut.

```
class Personne:
    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("L'âge ne peut pas être négatif.")
        self._age = value
```

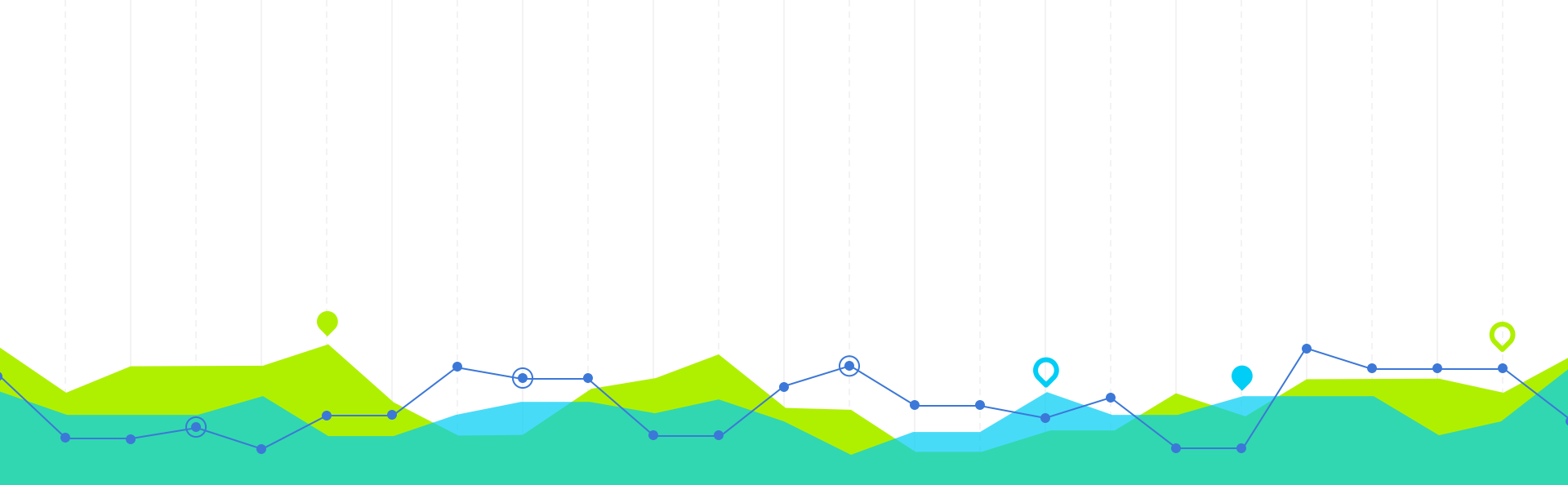
Pour aller plus loin

- <https://python.doctor/page-apprendre-programmation-orientee-objet-poo-classes-python-cours-debutants>
- <https://datascientest.com/programmation-orientee-objet-guide-ultime>
- <https://python.developpez.com/cours/apprendre-python-3/?page=la-programmation-orientee-objet>
- <https://kinsta.com/fr/blog/programmation-orientee-objet-python/>
- <https://courspython.com/classes-et-objets.html>
- <https://realpython.com/courses/python-class-inheritance/> (payant)

```
python3
~ python3
Python 3.7.6 (default, Dec 30 2019, 19:38:26)
[Clang 11.0.0 (clang-1100.0.33.16)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>> |
```





3. Capitaliser et partager son code avec GIT



Qu'est ce que Git ?

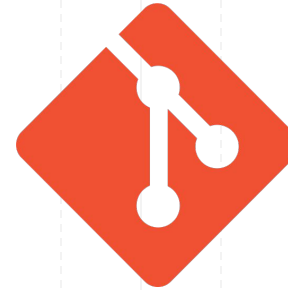
=> Système de gestion de versions distribué

Il permet à plusieurs personnes de travailler sur un même projet sans interférer les uns avec les autres.

outil essentiel pour **suivre l'évolution d'un projet**, revenir à des versions précédentes, et gérer les conflits lors de modifications simultanées.

Importance de Git en entreprise

- **collaboration** : Permet à plusieurs développeurs de travailler sur un même projet simultanément.
- **traçabilité** : Chaque modification est tracée : qui a fait quoi, quand.
- **sécurité** : Possibilité de revenir à n'importe quelle version précédente d'un projet.
- **agilité** : Facilite la gestion de projets agiles avec des fonctionnalités développées en parallèle.



git

Qu'est ce que Git ?



Git vs Plateformes basées sur Git

- **Git** : l'outil en ligne de commande, le cœur de la gestion de versions.
- **GitHub / GitLab / Bitbucket** : plateformes avec interface graphique et fonctionnalités supplémentaires pour la gestion de projets, la revue de code, l'intégration et le déploiement continu, etc.

L'importance de bien comprendre Git

La majeure partie des entreprises du secteur technologique utilise Git. Connaître Git est souvent une **compétence requise** pour les postes de développement.

Une utilisation incorrecte de Git peut entraîner des problèmes majeurs, comme la perte de code.



Configuration de Git



Configurer le nom et l'e-mail

Chaque commit dans Git contient le **nom** et l'**e-mail** de l'auteur :

```
git config --global user.name "Votre Nom"  
git config --global user.email "votre.email@example.com"
```

Il est possible de configurer un nom et un e-mail spécifiques à un projet particulier en supprimant le flag `--global` et en exécutant les commandes depuis le répertoire du projet concerné.

Configurer un éditeur par défaut

Git ouvrira souvent un éditeur de texte pour que vous saisissiez certains messages (comme les messages de commit). Si vous ne spécifiez pas d'éditeur, Git utilisera l'éditeur par défaut du système, souvent **Vim**.

Pour configurer, par exemple, Nano comme éditeur par défaut :

```
git config --global core.editor nano
```



Configuration de Git

Configuration des clés SSH pour une authentification sans mot de passe

SSH est un moyen sécurisé d'accéder à des ressources à distance.

Vous pouvez utiliser SSH pour interagir avec des **dépôts distants** sans avoir à vous logger à chaque fois.

Générez une nouvelle clé SSH si vous n'en avez pas déjà une :

```
ssh-keygen -t rsa -b 4096 -C "votre.email@example.com"
```

Assurez-vous que le ssh-agent est en cours d'exécution :

```
eval "$(ssh-agent -s)"
```

Ajoutez votre clé SSH privée à l'agent :

```
ssh-add ~/.ssh/id_rsa
```

Copiez la clé publique et ajoutez-la à votre compte sur des plateformes comme GitHub, GitLab, etc.

```
cat ~/.ssh/id_rsa.pub
```

Qu'est ce qu'un dépôt (ou repository) git ?



C'est un dossier contenant un projet sur lequel on souhaite suivre les modifications

Initialiser un nouveau dépôt Git

```
git init
```

Cette commande crée un nouveau sous-dossier **.git** dans votre dossier actuel, qui contient tous les **métadonnées** nécessaires pour le suivi de version.



Clonage d'un repo



Quand et pourquoi cloner ?

Vous souhaitez avoir une **copie locale** d'un projet hébergé en ligne (GitHub, GitLab ou Bitbucket) pour pouvoir travailler dessus.

```
git clone <URL>
```

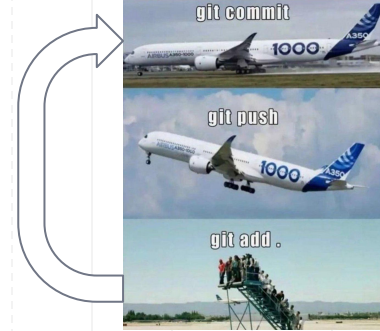
La commande `git clone` copie le dépôt, son historique, ses branches et toutes ses métadonnées sur votre ordinateur.

Clonage via HTTPS et SSH : en entreprise, l'utilisation de **SSH est préférée** pour des raisons de sécurité et de commodité.

Après avoir cloné un dépôt, il est courant de

- **Vérifier la branche** sur laquelle vous vous trouvez avec `git branch`
- Utiliser `git log` pour **voir les derniers commits**
- Si vous travaillez en collaboration, utiliser `git pull` pour s'assurer d'**avoir les dernières mises à jour** du dépôt distant

Fondamentaux du cycle de vie du fichier

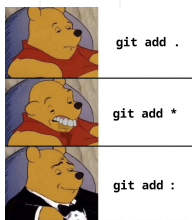


Les trois états des fichiers

- **Modified** (Modifié) : si le fichier a été modifié depuis le dernier commit
- **Staged** (Indexé) : quand vous avez décidé que les modifications apportées à un fichier doivent être conservées pour le prochain commit. Les fichiers dans cet état sont prêts à être "commités" (enregistrés dans l'historique de Git).
- **Committed** (Engagé) : L'état final, vos modifications ont été enregistrées dans la base de données locale de Git.

Utilisation de `git status` pour voir l'état actuel des fichiers

- Commande essentielle pour savoir où en sont vos fichiers
- Affiche les fichiers modifiés, indexés, non suivis, etc
- Fournit souvent des conseils sur ce qu'il faut faire ensuite



Fondamentaux du cycle de vie du fichier

Cas de fichiers modifiés

Si des fichiers ont été modifiés mais pas indexés, ils seront listés sous la rubrique "Changes not staged for commit" :

```
(base) → repo_git_test git:(master) ✖ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   mon_fichier.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Cas de nouveaux fichiers

Si des fichiers ont été créés et pas encore suivis par git, ils seront listés sous la rubrique "Untracked files" :

```
(base) → repo_git_test git:(master) ✖ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        mon_fichier.py

nothing added to commit but untracked files present (use "git add" to track)
```



git add .

git add *

git add :

Fondamentaux du cycle de vie du fichier

Modified à Staged : Indexation des modifications avec git add

git add <fichier> pour indexer un fichier.

Vous dites à Git quelles modifications (dans quels fichiers) vous souhaitez inclure dans le prochain commit.

Staged à Committed : Enregistrement des modifications avec git commit

git commit pour enregistrer les modifications de tous les fichiers indexés dans l'historique.

Ceci créera un **nouveau commit** dans votre historique avec un identifiant unique.

```
(base) → repo_git_test git:(master) ✗ git add mon_fichier.py
(base) → repo_git_test git:(master) ✗ git status
On branch master
```

No commits yet

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   mon_fichier.py
```

```
(base) → repo_git_test git:(master) ✗ git commit -m "Description pertinente des modifications"
[master (root-commit) 5f9c6e2] Description pertinente des modifications
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 mon_fichier.py
(base) → repo_git_test git:(master) git status
On branch master
nothing to commit, working tree clean
```

Les commits



1. git commit



2. git push



3. git out!

Astuces pour une meilleure gestion des commits

- le message de commit doit être concis mais descriptif. En entreprise, cela aide les collègues (et vous-même plus tard) à comprendre pourquoi un changement a été fait.
- utiliser `git diff` avant un `git add` pour voir quelles modifications seront indexées.
- si vous réalisez que vous avez oublié quelque chose après un commit, mais que vous n'avez pas encore poussé vos changements, vous pouvez utiliser `git commit --amend` pour **modifier le dernier commit**.
- `git stash` qui permet de "mettre de côté" des modifications temporaires afin de basculer vers une autre branche ou de réaliser une autre tâche, puis de les récupérer plus tard.



Navigation dans l'historique

Visualiser l'historique des commits

`git log` : affiche l'historique des commits du plus récent au plus ancien

Options utiles :

`git log --oneline` : affiche chaque commit sur une seule ligne, idéal pour avoir un aperçu rapide

`git log -n <nombre>` : limite l'affichage à un certain nombre de commits

`git log --graph` : affiche une représentation graphique des branches et merges

Examiner les changements d'un commit spécifique

`git show <SHA>` : Affiche les modifications apportées dans un commit spécifique, identifié par son SHA (ou les premiers caractères de celui-ci). C'est utile pour voir exactement ce qui a changé dans un commit donné



Navigation dans l'historique



Se déplacer entre les commits ou les branches

`git checkout <SHA>` : permet de **retourner** à un état précis du code basé sur le SHA du commit. C'est utile pour explorer l'historique ou retrouver une ancienne version d'un fichier.

`git checkout <nom_branche>` : bascule vers une branche spécifique.

Réinitialiser votre espace de travail

`git reset` # Utile si vous voulez annuler certains changements.

Différents modes :

`git reset --soft <SHA>` # Réinitialise HEAD à un certain commit, mais conserve les modifications dans la zone de staging.

`git reset --mixed <SHA>` # (par défaut) : Réinitialise HEAD et désindexe les modifications.

`git reset --hard <SHA>` # Réinitialise HEAD et supprime toutes les modifications dans votre espace de travail.

faire attention avec certaines commandes (comme `git reset --hard`), car elles peuvent supprimer des modifications.



Les branches

Une branche est comme une **version parallèle du code**, permettant de travailler sur différentes fonctionnalités ou corrections **sans affecter la branche principale** (souvent appelée master ou main).

Création et navigation entre les branches

`git branch` : voir toutes les branches locales

`git switch -c [nom_de_la_branche]` : créer une nouvelle branche et s'y déplacer

`git switch [nom_de_la_branche]` : se déplacer vers une branche existante

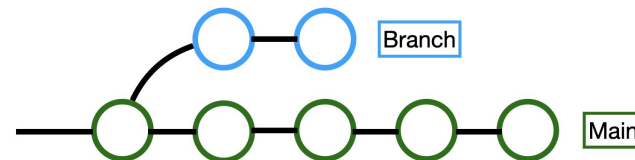
Suppression d'une branche

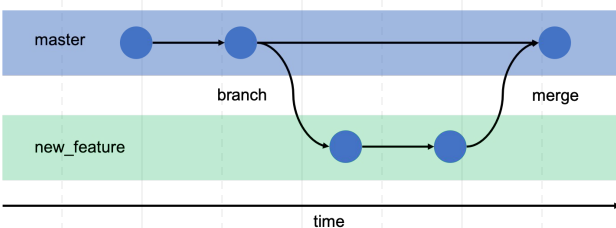
`git branch -d [nom_de_la_branche]`

Travail sur une branche

Faire des commits sur une branche **n'affecte pas** les autres branches.

Utile pour développer des fonctionnalités, corriger des bugs ou expérimenter.





Fusion de branche



Fusionner les branches

`git merge [nom_de_la_branche]` : fusionne la branche spécifiée dans la branche courante

Possibilité de **conflits** lors de la fusion : zones du code modifiées dans les deux branches.

Git ne sait pas quelle version garder.

Résolution des conflits : Git marque les zones problématiques dans les fichiers.

L'utilisateur doit les éditer manuellement pour **résoudre le conflit**, puis continuer le processus de fusion.

En entreprise, avant de fusionner une branche dans la branche principale, il est courant de procéder à une **revue de code** (via des **Pull/Merge Requests** sur des plateformes comme GitHub/GitLab).

Cela garantit la qualité du code, facilite la détection d'erreurs et partage la connaissance au sein de l'équipe.



Interaction avec les dépôts distants

La collaboration en équipe et les conflits

- **Conflits**

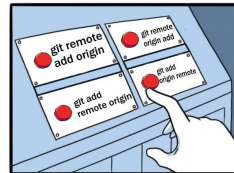
Quand deux personnes modifient la même partie d'un fichier en même temps, Git ne peut pas décider laquelle des deux modifications garder. **C'est au développeur de résoudre le conflit :**

- visualisation des conflits avec `git status`
- résolution manuelle des conflits dans le fichier
- après la résolution, utilisez `git add [fichier]` pour indiquer que le conflit a été résolu, puis continuez le processus (par exemple, complétez la fusion avec `git merge` ou le rebase avec `git rebase`)

- **Conseils**

- Toujours récupérer (**puller**) les dernières modifications **avant** d'entamer un travail important.
- **Communiquer** avec l'équipe pour éviter de travailler sur les mêmes parties du projet simultanément.

Interaction avec les dépôts distants



Avec des projets hébergés sur des serveurs distants comme GitHub, GitLab ou Bitbucket.

Qu'est-ce qu'un dépôt distant ?

Un dépôt distant est une version de votre projet hébergée sur Internet ou sur un réseau. Vous pouvez avoir plusieurs dépôts distants avec lesquels vous pouvez interagir.

Commandes principales :

```
git remote add [nom] [url] # ajouter un nouveau dépôt distant.  
git remote -v # voir la liste des dépôts distants.  
git remote remove [nom] # supprimer un dépôt distant.
```

When you're dead but remember you forgot to git commit git push your last code iterations



Interaction avec les dépôts distants

Partager les modifications avec le dépôt distant

pull (Récupérer)

Récupérer les mises à jour du dépôt distant et les fusionner avec votre travail actuel.

```
git pull [nom_du_remote] [nom_de_la_branche] # ex : git pull origin master
```

fetch

Récupère les mises à jour du dépôt distant sans les fusionner avec votre travail. Utile pour voir ce que les autres ont fait avant de décider de fusionner.

```
git fetch [nom_du_remote] # ex : git fetch origin
```

différence avec pull:

git pull = git fetch + git merge.

Autrement dit, pull fait un fetch et essaie ensuite de fusionner les changements récupérés avec votre travail actuel.

push (Pousser)

Envoyer vos commits vers un dépôt distant. C'est la manière de partager vos changements avec d'autres collaborateurs.

```
git push [nom_du_remote] [nom_de_la_branche] # ex : git push origin master
```



Bonnes pratiques en entreprise

Commits Atomiques

Chaque commit doit représenter une unité logique de changement. Cela facilite la lecture de l'historique, le débogage et la révision.

Messages de commit clairs

Le titre du commit doit être concis et direct (généralement moins de 50 caractères).

Éviter les commits "temporaires"

Des messages comme "test", "correction rapide", etc., ne sont pas informatifs.

Utiliser des branches

Au lieu de travailler sur la branche principale, créez une nouvelle branche pour chaque nouvelle fonctionnalité ou correctif.

pull/merge Requests (PR/MR)

Lorsqu'une fonctionnalité ou un correctif est prêt à être mergé à la branche principale, créez un PR/MR.

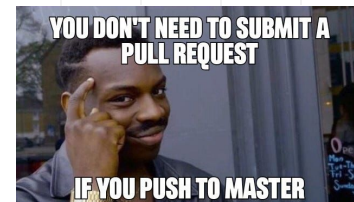
Cela offre l'opportunité à l'équipe de revoir les modifications avant leur intégration, garantissant ainsi la qualité du code.

Gérer les conflits rapidement et avec soin

Les conflits peuvent survenir lorsque plusieurs personnes travaillent **sur le même code**.

Garder son répertoire local à jour

Utilisez `git pull` ou `git fetch` régulièrement pour vous assurer d'avoir les dernières modifications.



Comment relire / commenter une MR ?

1. Comprendre le contexte

Lisez la description de la MR. Assurez-vous de comprendre ce que la MR est censée accomplir et pourquoi

2. Vérifier le respect des conventions

Le code respecte-t-il les conventions de codage / standards de l'équipe ou du projet ? Les messages de commit sont-ils clairs ?

3. Examinez la structure générale

Est-ce que le code est bien structuré ? Est-il modulaire et facilement maintenable ? Duplications ? Fichiers au bon endroit ?

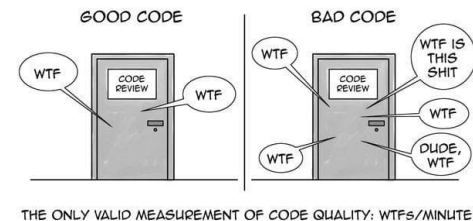
4. Examen détaillé du code

- lisez le code ligne par ligne. Assurez-vous de comprendre chaque changement
- vérifiez la logique. Y a-t-il des bugs potentiels ? Le code gère-t-il correctement les cas d'erreur?
- assurez-vous que les tests ont été changés et couvrent bien les nouvelles fonctionnalités ou les changements

5. Vérifier la sécurité

Vérifiez qu'il n'y a pas de code potentiellement dangereux, comme des injections SQL, des expositions de données sensibles ou des problèmes d'authentification. Assurez-vous que les dépendances ajoutées ne présentent pas de vulnérabilités connues.

Comment relire / commenter une MR ?



6. Performance et optimisation

Y a-t-il des endroits où le code pourrait être inefficace ou lent ?

7. Commenter

- Soyez précis dans vos commentaires. Si vous pointez un problème, essayez de **suggérer une solution**.
- Soyez constructif et **évitez les commentaires négatifs** ou vagues.
- **Posez des questions** si quelque chose n'est pas clair plutôt que de supposer.
- Si vous aimez un aspect particulier du code ou si vous constatez une amélioration significative, faites-le savoir !

8. Tester manuellement

Si possible, vérifiez la fonctionnalité en local ou dans un environnement de pré-production. Testez divers scénarios d'utilisation.

9. Synthèse

Si vous avez de nombreux commentaires, résumez les points principaux ou les préoccupations dans un commentaire général.

10. Communication

- Rappelez-vous que derrière chaque MR il y a un humain.
- Soyez respectueux, ouvert à la discussion et prêt à collaborer pour améliorer le code.
- Encouragez une culture de feedback constructif et d'apprentissage continu.

La revue de code s'améliore avec la pratique et l'expérience.
Plus vous le faites, meilleur vous serez !

Conclusion



Git est une **compétence fondamentale** pour tout développeur, particulièrement en environnement professionnel.

L'importance de la pratique régulière

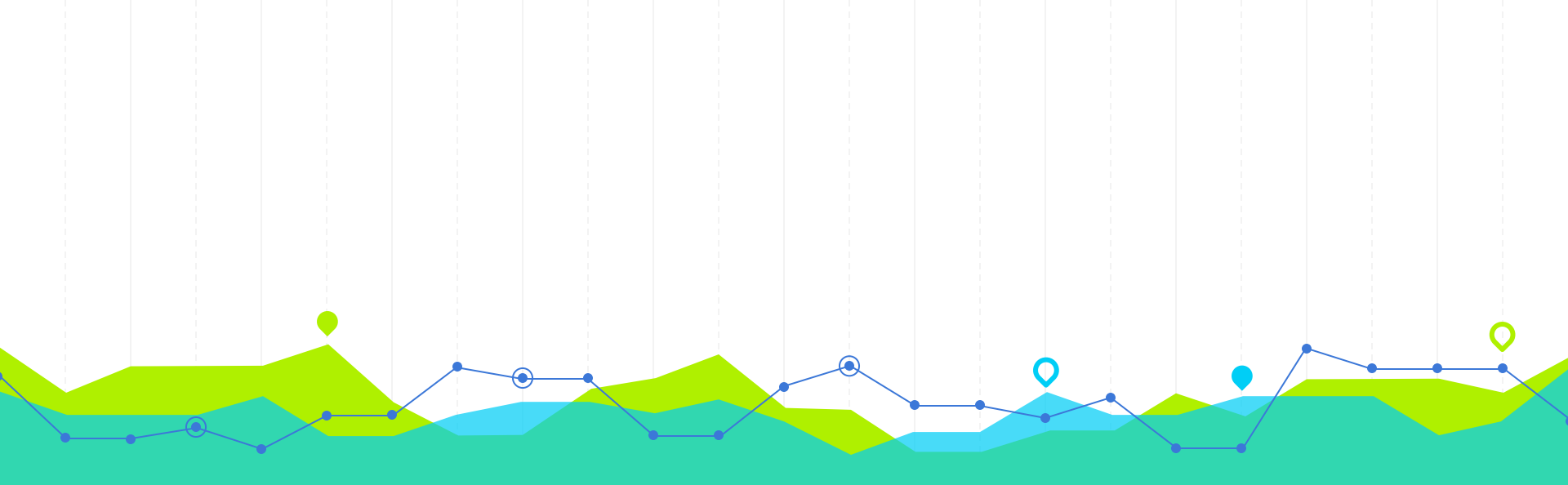
La maîtrise de Git vient avec la pratique. Il est essentiel de l'utiliser régulièrement, de créer des projets personnels ou de contribuer à des projets open-source pour renforcer cette compétence.

Chaque erreur ou conflit rencontré est une **opportunité d'apprendre**.

La poursuite de l'apprentissage

Il reste beaucoup à découvrir sur Git : rebase, cherry-pick, bisect et bien d'autres commandes avancées.

Pour vous entraîner: <https://learngitbranching.js.org>



4. Savoir utiliser les fonctionnalités de GitHub / GitLab

Fonctionnalités de GitHub/GitLab



Création d'un dépôt (repository)

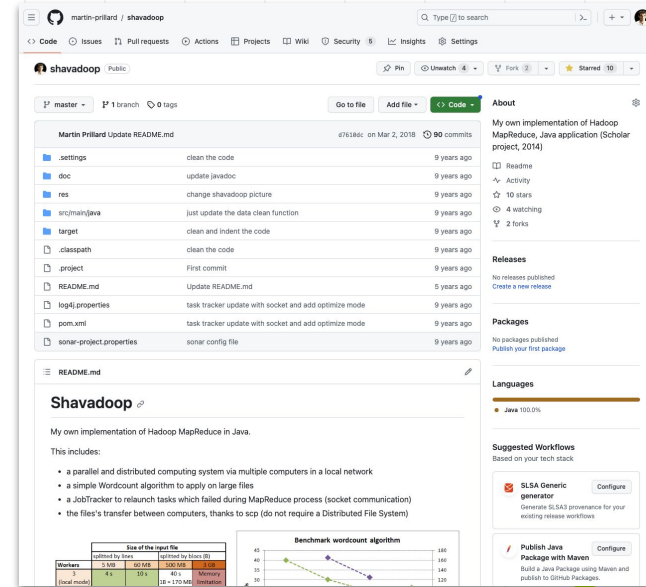
- cliquez sur le bouton **+** ou **New** (selon la plateforme) dans le coin supérieur droit pour **créer un nouveau dépôt**.
- Donnez-lui un nom, une description, choisissez s'il doit être public ou privé, et initialisez-le avec un README.

Cloner ou télécharger un dépôt

- Cloner le dépôt en utilisant l'URL fournie **ou** télécharger le dépôt sous forme de fichier ZIP

Naviguer dans le dépôt

- La page principale d'un dépôt vous montre la liste des fichiers, le README s'il existe, et une barre d'onglets pour accéder à Issues, Pull Requests, Actions, etc.
- Vous pouvez cliquer sur les fichiers pour voir leur contenu ou leur historique de modifications.



Fonctionnalités de GitHub/GitLab



Commit

- Naviguez vers le fichier que vous souhaitez modifier
- Faites vos modifications, puis descendez pour entrer un **message de commit** et cliquez sur **Commit changes**

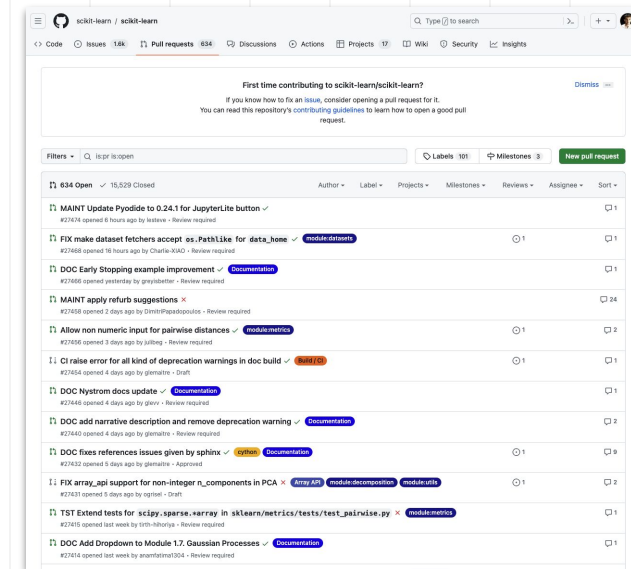
Issues & Merge/Pull Requests

- Utilisez l'onglet **Issues** pour **signaler des bugs**, **proposer des améliorations** ou **discuter** de certaines tâches.
- Utilisez l'onglet **Pull Requests** (GitHub) ou **Merge Requests** (GitLab) pour **proposer des modifications** au code qui seront éventuellement fusionnées avec la branche principale.

Gestion des branches

- **changer de branche** via le menu déroulant situé généralement en haut à gauche de la liste des fichiers.
- **créer une nouvelle branche** en entrant le nom de votre nouvelle branche

Exemple : <https://github.com/scikit-learn/scikit-learn/pull/27409>



Fonctionnalités de GitHub/GitLab



Fusionner des branches

- Une fois que vous avez apporté des modifications dans une branche, vous pouvez ouvrir une **Pull/Merge Request** pour **fusionner** ces modifications dans une autre branche.
- Si des **conflits** se présentent, GitHub/GitLab vous indiquera les fichiers en conflit et vous permettra de les **résoudre** directement via l'interface web.

Actions & CI/CD

- Les deux plateformes proposent des solutions pour l'**intégration continue** et le **déploiement continu**.
- Sur GitHub, c'est avec "GitHub Actions". Sur GitLab, c'est avec les "GitLab CI/CD Pipelines".

Gestion des accès

- Vous pouvez **inviter** des collaborateurs, définir leurs **niveaux d'accès** ou gérer l'accès public/privé de votre dépôt.

Wiki & Pages

- Vous pouvez utiliser le **Wiki** pour **documenter** votre projet.
- GitHub et GitLab proposent des solutions pour **héberger** des pages web directement depuis votre dépôt.



À votre tour

TD Git

Objectif : utiliser les commandes de base de Git nécessaires pour travailler efficacement en équipe dans un contexte d'entreprise.

Prérequis

- Avoir Git installé sur sa machine. Si ce n'est pas le cas, consultez [la documentation officielle](#) pour l'installer.
- Avoir un compte GitHub ou GitLab.

Exercice 1 : Configuration initiale

Ouvrez un terminal ou une invite de commande et configurez votre nom et votre adresse e-mail:

```
git config --global user.name "Votre Nom"  
git config --global user.email "votre@email.com"
```

TD Git

Exercice 2 : Créer un nouveau dépôt et faire son premier commit

1. Créez un nouveau dossier pour votre projet:
`mkdir td_git`
`cd td_git`
2. Initialisez un nouveau dépôt Git:
`git init`
3. Créez un fichier `README.md` et écrivez "Mon premier projet avec Git" dedans.
4. Ajoutez ce fichier à l'index:
`git add README.md`
5. Faites un commit avec un message descriptif:
`git commit -m "Ajout du fichier README"`

TD Git

Exercice 3 : Travailler avec des branches

1. Créez une nouvelle branche appelée **develop**:
`git checkout -b develop`
2. Modifiez le fichier **README.md** et ajoutez une ligne: "Ceci est la branche de développement."
3. Committez vos changements:
`git add README.md`
`git commit -m "Mise à jour du README pour la branche develop"`
4. Revenez à la branche **master**:
`git checkout master`
5. Fusionnez la branche **develop** dans **master**:
`git merge develop`

TD Git

Exercice 4 : Collaboration avec un dépôt distant

1. Créez un nouveau dépôt sur GitHub ou GitLab. Ne pas initialiser le dépôt avec un README, .gitignore ou licence.
2. Liez votre dépôt local à ce dépôt distant:
`git remote add origin URL_DU_DEPOT`
3. Envoyez vos changements sur le dépôt distant:
`git push -u origin master`

Exercice 5 : Récupérer des changements

1. Imaginez qu'un collègue ait pushé des modifications sur le dépôt distant. Récupérez les changements depuis le dépôt distant :
`git pull origin master`
Si vous rencontrez des conflits lors du `pull`, résolvez-les en éditant les fichiers concernés, puis committez les changements.

Exercice 6 : Utilisation de .gitignore

1. Créez un fichier `temp.txt` dans votre projet.
2. Créez un fichier `.gitignore` et ajoutez-y la ligne `temp.txt`.
3. Faites un commit du fichier `.gitignore`.
4. Vérifiez que `temp.txt` n'est pas suivi par Git.